

Instructor Handout 6: Hardwired Control Unit Design

PCC CS-401: Computer Organization and Architecture

August 7, 2026

Instructor Copy — Not for Student Distribution

6.1 Before You Teach This Lecture

- **Mano, *Computer System Architecture*, 3rd ed., Sec. 5-4 “Timing and Control,” p. 135, with Fig. 5-6 on p. 137 and Fig. 5-7 on p. 138** (indexed, page-cited). Fig. 5-7 is a literal clock/timing waveform diagram (not a block diagram) — the lecture’s slides describe SC’s timing in prose only, so if you want to sketch actual clock pulses and T_i waveforms on the board, this figure is worth having open.
- **Mano, Sec. 5-9 “Design of Basic Computer,” pp. 157–164 (control logic gates, control of registers/common bus), and Sec. 5-10, pp. 164–167 (accumulator control logic)** (indexed). Figures 5-16/5-17/5-18 in this range are the book’s own real gate-level realizations (control gates for AR, IEN, and bus selection) — a third worked AND-OR example beyond ADD/SUB’s Z_{in} , if a student wants to see the pattern applied to a register-control signal instead of a data-path signal.
- **Hamacher, Ch. 7** — *not yet page-indexed* for this course. Chapter-level citation only.
- **Reminder:** this lecture’s anchor reading was corrected this term from a prior mis-citation of Mano Ch. 7 (Microprogrammed Control, actually next week’s topic) to Mano Ch. 5. If any of your own older notes/slides for this course still say “Mano Ch. 7” for hardwired control, they are citing the wrong chapter — see the Mano1993 textbook index’s correction record, at [master_supporting_docs/CS401/supporting_books/Mano1993/index.md](#), for the full history.

6.2 Extra Worked Examples

6.2.1 Three Instructions Sharing a Signal

The student Notes derive Z_{in} for two instructions (ADD, SUB). Extend to three: suppose AND also uses Z at its own T_2 .

$$Z_{in} = (D_{ADD} \cdot T_2) + (D_{SUB} \cdot T_2) + (D_{AND} \cdot T_2) = T_2 \cdot (D_{ADD} + D_{SUB} + D_{AND}).$$

Instructor note: worth showing that the AND-OR gate realization simply grows by one more AND gate feeding the same OR gate — the OR gate itself does not need to change shape, only gain an input, which is a good moment to point at real hardware’s fixed-fan-in OR gates as a practical limit on how many instructions can cheaply share one signal.

6.2.2 A One-Step Instruction

Not every instruction needs three steps. Consider MOV R1, R2 (register-to-register copy, no ALU operation):

Step	RTL	Control signals active
T_1	$R1 \leftarrow [R2]$	$R2_{out}, R1_{in}, SC_{clear}$

Boolean equations: $R2_{out} = D_{MOV} \cdot T_1$, $R1_{in} = D_{MOV} \cdot T_1$, $SC_{clear} = D_{MOV} \cdot T_1$. **Instructor note:** useful as a contrast case — it shows the state-table method scales *down* as cleanly as it scales up, and that SC_{clear} can co-occur with T_1 when an instruction is only one step long, which some students find counterintuitive after seeing only 3-step examples.

6.3 Teaching Notes

Instructional suggestions, not fact-checked content — adapt freely.

6.3.1 Control Unit Functions

- **Common misconception:** students often picture D_i as a fixed wire with a fixed name across the whole course, rather than understanding it is *one output among many* from a single decoder, exactly one of which is high at a time. Reinforce “one-hot” explicitly and early.

6.3.2 The Sequence Counter and Timing Generation

- **Common error:** when deriving equations, students frequently forget that SC_{clear} itself needs an opcode-and-step term ($D_{ADD} \cdot T_3$), treating it as a special signal exempt from the general pattern. It is not — it is generated by the exact same method as every other control signal.
- **Board sequence:** for the SC + decoder diagram, draw SC and the decoder box first, then add the T_i output lines one at a time as you narrate “one line goes high per cycle,” rather than showing all outputs pre-drawn.

6.3.3 Opcode Decoding and the Control Matrix

- **Board sequence:** build the two-decoders diagram left to right, matching data flow — IR/SC first, then the two decoders, then the control matrix, then outputs last. Drawing the control matrix box before its inputs tends to make the diagram read as “a black box that magically knows the answer” rather than “a circuit built from two decoders.”
- **Live question:** “If we add a 4-step MUL instruction, does SC’s bit width need to change?” (A good check on whether students understand SC’s width is set by the *longest* instruction in the ISA, not by any one instruction.)

6.3.4 Deriving the Control Equations

- **Pacing:** the state-table $\rightarrow$ Boolean-equations $\rightarrow$ gates chain is usually the part of this lecture that benefits most from a second, slower pass, ideally with the class supplying the equation for one new signal themselves (e.g. $R1_{in}$) before you reveal it.